Enterprise Zero Trust Security Architecture

HL-CYB-001 · Advanced cybersecurity engineering lab

1. Project Overview

This project designs an isolated enterprise lab where people receive access to the systems needed for their role. It brings identity, network restrictions, certificates, logging, and recovery into one testable engineering assignment.

The scenario uses synthetic Purple Eye users and data, with no production or home-network connection.

2. Business Problem

An employee may need to read a report without changing the application or administering its server. Remote access must let that employee reach the approved resource without opening the whole network.

The lab will demonstrate allowed and denied actions, access revocation, and the evidence behind each result. It makes no production-impact or compliance-certification claim.

3. Project Objectives

- Design separate security zones and a protected administrative path using the available hardware.
- Give synthetic users and devices only the resource access required for their role.
- Test certificate trust, remote access, logging, automation, and recovery in individually approved stages.
- Publish reviewed engineering evidence and accurate task reports.

4. Architecture Overview

Proxmox VE will run the virtual machines; OPNsense will control traffic between Management, Identity, Endpoints, Services, Monitoring, and Remote access zones. Active Directory manages lab identities; host/application permissions control resource access. An offline certificate authority protects the root of certificate trust.

Internet access needs a separately approved isolation design. See the architecture for the diagram, capacity, and allowed flows, and the glossary for terms.

5. Technologies Used

The repository currently uses **Git, GitHub, Python, and GitHub Actions** for version control and documentation checks.

Planned lab technologies are Proxmox VE, OPNsense, Windows Server Evaluation, Active Directory, DNS, Group Policy, Linux, public key infrastructure (PKI), Tailscale, Twingate, Ansible, and PowerShell. Their presence in the design does not mean they are installed or configured.

6. Key Security Capabilities

Planned controls include access denied unless permitted, separate ordinary/admin identities, resource-scoped remote access, certificate issuance/revocation, central audit records, and encrypted backups with a timed restore. Each has a requirement and future test.

Production Microsoft 365 identities, PE-DAP, customer data, the corporate website, and production hosting, including AWS Lightsail, are excluded. Actual addresses, credentials, tenant details, raw logs, and backups stay outside public Git.

7. Implementation Status

Repository setup, planning documents, task reporting, and documentation validation have been implemented and checked. **Lab implementation is PLANNED; operational controls are NOT TESTED.** Owner user acceptance testing (UAT) remains pending.

T1 preserves the bootstrap results. Its reporting addendum records the reporting migration. T2 records the documentation review. No implementation stage starts without separate owner approval.

8. Testing and Validation

Use Python 3.11 or later with the pinned documentation dependencies. With that documentation environment active:

```
python scripts/validate_docs.py
python -m unittest discover -s tests -v
git diff --check
```

The checks cover documentation, local links, traceability, report history, and selected publication risks; they do not test the lab. Before completion, also run `python scripts/validate_docs.py --require-finalized`. Review public content manually: the scanner is not exhaustive, and CI does not fetch external references.

CI uses read-only permissions, a commit-pinned checkout, no retained credentials, and no lab access. The test plan defines future functional, denied-access, security, and recovery checks.

9. Project Documentation

Use the documentation library for source/PDF pairs, including the assignment PDF, lab guide PDF, and architecture PDF. Markdown remains editable; PDFs are generated with the documented Windows build.

- Assignment: business case, requirements, deliverables, and assessment rubric.
- Implementation guide: stages A-L, checks, rollback, and approval gates.
- Architecture and security requirements: boundaries, access rules, threats, and standards mappings.
- Acceptance criteria, decisions, and risks: evaluation and unresolved choices.
- Task reports and bootstrap evidence: work performed and evidence available.
- Project final report template, glossary, and references: project assessment, terminology, and authoritative sources.

10. Reporting and GitHub Workflow

Every task needs a sequential, finalized Markdown-only report (`reports/Ti-Report.md`) before engineering completion. Reports and addenda do not receive PDFs; the entire reporting directory is excluded from PDF generation. Existing report PDFs are archived artifacts. Finalized Markdown reports remain immutable; corrections use an addendum or later task report. Owner UAT remains separate.

Use a task branch, focused commits, CI, and a reviewed pull request before merging; then fast-forward local `main`. Synchronization is intentional Git activity. AGENTS.md defines the workflow, approvals, writing standards, and prohibition on background upload. Protection/reviewer settings remain owner decisions.

11. Author and Repository Information

Maintained under waqashamdani-ca in the public enterprise-zero-trust-security repository, with `main` as the default branch.

No software license has been selected. The next engineering task is an owner-approved, read-only hardware and isolation readiness assessment; this documentation work does not authorize it.